Lab 7 Solution

```
package alu_pkg;

    typedef enum logic [3:0] {
        ADD  = 4'h0,
        SUB  = 4'h1,
        MUL  = 4'h2,
        DIV  = 4'h3,
        AND  = 4'h4,
        OR   = 4'h5,
        NAND = 4'h6,
        NOR  = 4'h7,
        XOR  = 4'h8,
        XNOR = 4'h9,
        EQ   = 4'hA,
        GT   = 4'hB,
        LT   = 4'hC,
        SHR  = 4'hD,
        SHL  = 4'hE,
        INV  = 4'hF
    } alu_fun_e;

endpackage
```

Class:

```
import alu_pkg::*;

class stimulus;

    // ------------------------------------------------------------
    // class members
    // ------------------------------------------------------------
    rand alu_fun_e   ALU_FUN;
    rand logic [7:0] A;
    rand logic [7:0] B;
    rand logic       enable;
    rand logic       RST;
         bit         clk;

    // walking-ones position for A on OR/XOR -- NOT rand, just remembers
    // position between calls so each randomize() advances to the next bit
    bit [2:0]   idx;

    // ------------------------------------------------------------
    // 1. RST asserted with low probability
    // ------------------------------------------------------------
    constraint c_rst {
        RST dist { 1'b0 :=  5,
                   1'b1 := 95 };
    }

    // ------------------------------------------------------------
    // 2. enable: valid cases far more frequent than invalid
    // ------------------------------------------------------------
    constraint c_enable {
        enable dist { 1'b0 := 20,
                      1'b1 := 80 };
    }

    // ------------------------------------------------------------
    // 3. ALU_FUN: arithmetic > logical > comparison > shift > invalid
    // ------------------------------------------------------------
    constraint c_alufun {
        ALU_FUN dist { ADD  := 12,
                       SUB  := 12,
                       MUL  := 12,
                       DIV  := 12,
                       AND  :=  8,
                       OR   :=  8,
                       NAND :=  8,
                       NOR  :=  8,
                       XOR  :=  8,
                       XNOR :=  8,
                       EQ   :=  5,
                       GT   :=  5,
                       LT   :=  5,
                       SHR  :=  2,
                       SHL  :=  2,
                       INV  :=  1 };
    }
```

```
// ------------------------------------------------------------------
// 4. shifts must always have enable high
//    invalid opcode must always have enable low
// ------------------------------------------------------------------
constraint c_enable_fun {
    (ALU_FUN == SHR || ALU_FUN == SHL) -> (enable == 1'b1);
    (ALU_FUN == INV)                   -> (enable == 1'b0);
}

// ------------------------------------------------------------------
// 5. RST must never fire during MUL or DIV
// ------------------------------------------------------------------
constraint c_rst_fun {
    (ALU_FUN == MUL || ALU_FUN == DIV) -> (RST == 1'b1);
}

// ------------------------------------------------------------------
// 6a. ADD: boundary values MAXPOS/ZERO/MAXNEG hit more often
// ------------------------------------------------------------------
constraint c_ab_add {
    (ALU_FUN == ADD) -> {
        A dist { 8'h7F          := 5,
                 8'h00          := 5,
                 8'h80          := 5,
                 [8'h01:8'h7E] := 1,
                 [8'h81:8'hFF] := 1 };
        B dist { 8'h7F          := 5,
                 8'h00          := 5,
                 8'h80          := 5,
                 [8'h01:8'h7E] := 1,
                 [8'h81:8'hFF] := 1 };
    }
}

// ------------------------------------------------------------------
// 6b. MUL: same boundary emphasis as ADD
// ------------------------------------------------------------------
constraint c_ab_mul {
    (ALU_FUN == MUL) -> {
        A dist { 8'h7F          := 5,
                 8'h00          := 5,
                 8'h80          := 5,
                 [8'h01:8'h7E] := 1,
                 [8'h81:8'hFF] := 1 };
        B dist { 8'h7F          := 5,
                 8'h00          := 5,
                 8'h80          := 5,
                 [8'h01:8'h7E] := 1,
                 [8'h81:8'hFF] := 1 };
    }
}
```

```
    // ------------------------------------------------------------------
    // 7a. OR: A walks a single active bit (idx), B mostly zero
    // ------------------------------------------------------------------
    constraint c_ab_or {
        (ALU_FUN == OR) -> {
            A == (8'b1 << idx);
            B dist { 8'h00          := 8,
                     [8'h01:8'hFF] := 1 };
        }
    }

    // ------------------------------------------------------------------
    // 7b. XOR: same walking-ones / zero-B pattern as OR
    // ------------------------------------------------------------------
    constraint c_ab_xor {
        (ALU_FUN == XOR) -> {
            A == (8'b1 << idx);
            B dist { 8'h00          := 8,
                     [8'h01:8'hFF] := 1 };
        }
    }

    // note: SHR and SHL have no constraints on A or B

    // ------------------------------------------------------------------
    // fun_to_string: converts alu_fun_e enum to its name string
    // ------------------------------------------------------------------
    function string fun_to_string(alu_fun_e f);
        case (f)
            ADD:     return "ADD";
            SUB:     return "SUB";
            MUL:     return "MUL";
            DIV:     return "DIV";
            AND:     return "AND";
            OR:      return "OR";
            NAND:    return "NAND";
            NOR:     return "NOR";
            XOR:     return "XOR";
            XNOR:    return "XNOR";
            EQ:      return "EQ";
            GT:      return "GT";
            LT:      return "LT";
            SHR:     return "SHR";
            SHL:     return "SHL";
            INV:     return "INV";
            default: return "???";
        endcase
    endfunction

    // ------------------------------------------------------------------
    // pre_randomize: show values before the solver runs
    // ------------------------------------------------------------------
    function void pre_randomize();
        $display("\n ********** BEFORE  RANDOMIZATION ************");
        $display("     RST      =  %0b",     RST);
        $display("     enable   =  %0b",     enable);
        $display("     ALU_FUN  =  %s",      fun_to_string(ALU_FUN));
        $display("     A        =  8'h%02h", A);
        $display("     B        =  8'h%02h", B);
    endfunction
```

```
    // ------------------------------------------------------------------
    // post_randomize: show values after the solver finishes
    // ------------------------------------------------------------------
    function void post_randomize();
        if(ALU_FUN == 4'h5 || ALU_FUN == 4'h8)
            idx = (idx + 1) % 8;        // advance walking-one position for next call

        $display("\n ********** AFTER   RANDOMIZATION ************");
        $display("     RST     =  %0b",     RST);
        $display("     enable  =  %0b",     enable);
        $display("     ALU_FUN =  %s",      fun_to_string(ALU_FUN));
        $display("     A       =  8'h%02h", A);
        $display("     B       =  8'h%02h", B);
    endfunction

    // ------------------------------------------------------------------
    // drive: apply inputs at negedge, wait posedge for DUT to latch
    // ------------------------------------------------------------------
    task drive(ref logic [7:0] dut_A,
               ref logic [7:0] dut_B,
               ref logic [3:0] dut_FUN,
               ref logic       dut_en,
               ref logic       dut_RST);
        @(negedge clk);
        dut_A   = A;
        dut_B   = B;
        dut_FUN = logic'(ALU_FUN);
        dut_en  = enable;
        dut_RST = RST;
        $display("  [drive]  RST=%0b  en=%0b  FUN=%s  A=8'h%02h  B=8'h%02h",
                 RST, enable, fun_to_string(ALU_FUN), A, B);
        @(negedge clk);
    endtask

    // ------------------------------------------------------------------
    // print helpers
    // ------------------------------------------------------------------
    function void print_rand_mode();
        $display("  ALU_FUN.rand_mode = %0d", ALU_FUN.rand_mode());
        $display("  A.rand_mode       = %0d", A.rand_mode());
        $display("  B.rand_mode       = %0d", B.rand_mode());
        $display("  enable.rand_mode  = %0d", enable.rand_mode());
        $display("  RST.rand_mode     = %0d", RST.rand_mode());
    endfunction

    function void print_constraint_mode();
        $display("  c_rst         = %0d", c_rst.constraint_mode());
        $display("  c_enable      = %0d", c_enable.constraint_mode());
        $display("  c_alufun      = %0d", c_alufun.constraint_mode());
        $display("  c_enable_fun  = %0d", c_enable_fun.constraint_mode());
        $display("  c_rst_fun     = %0d", c_rst_fun.constraint_mode());
        $display("  c_ab_add      = %0d", c_ab_add.constraint_mode());
        $display("  c_ab_mul      = %0d", c_ab_mul.constraint_mode());
        $display("  c_ab_or       = %0d", c_ab_or.constraint_mode());
        $display("  c_ab_xor      = %0d", c_ab_xor.constraint_mode());
    endfunction

endclass
```

Testbench:

```
`include "alu_pkg.sv"
`include "stimulus_class.sv"
import alu_pkg::*;
module top;
    // ------------------------------------------------------------------
    // signals
    // ------------------------------------------------------------------
    bit           clk;
    logic         RST;
    logic [7:0]   A, B;
    logic [3:0]   ALU_FUN;
    logic         enable;
    logic         ALU_OUT_VALID;
    logic [15:0] ALU_OUT;

    // stimulus object
    stimulus stim;
    // ------------------------------------------------------------------
    // DUT instantiation
    // ------------------------------------------------------------------
    ALU dut (
        .CLK          (clk),
        .RST          (RST),
        .A            (A),
        .B            (B),
        .ALU_FUN      (ALU_FUN),
        .enable       (enable),
        .ALU_OUT_VALID(ALU_OUT_VALID),
        .ALU_OUT      (ALU_OUT)
    );
    // ------------------------------------------------------------------
    // clock generation - 10ns period
    // ------------------------------------------------------------------
    initial begin
        clk = 0;
        forever begin
            #5ns clk = ~clk;
            stim.clk = clk;
        end
    end
    // ------------------------------------------------------------------
    // golden_model: compute expected ALU_OUT for self-checking
    // (lives in the checker/tb side, not the stimulus class)
    // ------------------------------------------------------------------
    function automatic logic [15:0] golden_model(
        input logic [7:0] a,
        input logic [7:0] b,
        input logic [3:0] fun,
        input logic       en
    );
        if (!en)
            return 16'h0000;
        case (fun)
            4'h0: return a + b;
            4'h1: return a - b;
            4'h2: return a * b;
            4'h3: return (b == 0) ? 16'hFFFF : (a / b);
            4'h4: return a  &  b;
            4'h5: return a  |  b;
            4'h6: return ~(a & b);
            4'h7: return ~(a | b);
            4'h8: return  a  ^  b;
            4'h9: return ~(a ^  b);
            4'hA: return (a == b) ? 'hA : 0;
            4'hB: return (a >  b) ? 'hB : 0;
            4'hC: return (a <  b) ? 'hC : 0;
            4'hD: return a >> 1;
            4'hE: return a << 1;
            default: return 16'h0000;
        endcase
    endfunction
```

```
// ------------------------------------------------------------
// self-check task
// ------------------------------------------------------------
task automatic check(
    input logic [7:0]  a,
    input logic [7:0]  b,
    input logic [3:0]  fun,
    input logic        en,
    input logic        rst
);
    logic [15:0] exp;
    if (!rst || !en) return;
    exp = golden_model(a, b, fun, en);
    if (ALU_OUT === exp)
        $display("    [PASS]  %s  A=%02h B=%02h  OUT=%04h",
                 stim.fun_to_string(alu_fun_e'(fun)), a, b, ALU_OUT);
    else
        $display("    [FAIL]  %s  A=%02h B=%02h  OUT=%04h  EXP=%04h  ***",
                 stim.fun_to_string(alu_fun_e'(fun)), a, b, ALU_OUT, exp);
endtask

// ------------------------------------------------------------
// main test sequence
// ------------------------------------------------------------
initial begin

    stim = new();

    // initial reset
    RST     = 0;
    enable  = 0;
    A       = 0;
    B       = 0;
    ALU_FUN = 0;
    $display("\n[TB] reset asserted\n");
    repeat (3) @(posedge clk);
    RST = 1;
    @(posedge clk);
    $display("[TB] reset released\n");

    // ------------------------------------------------------
    // Req 1-5  baseline: all constraints active
    // ------------------------------------------------------
    $display("--- Req 1-5: constrained randomization + drive ---\n");

    repeat (30) begin
        assert(stim.randomize()) else $fatal(1, "randomize() failed");
        stim.drive(A, B, ALU_FUN, enable, RST);
        check(A, B, ALU_FUN, enable, RST);
    end

    // ------------------------------------------------------
    //  All constraints OFF
    // ------------------------------------------------------
    $display("\n--- All constraints OFF ---\n");

    stim.constraint_mode(0);
    stim.print_constraint_mode();

    repeat (10) begin
        assert(stim.randomize()) else $fatal(1, "randomize() failed");
        stim.drive(A, B, ALU_FUN, enable, RST);
        check(A, B, ALU_FUN, enable, RST);
    end

    // ------------------------------------------------------
    //  All constraints back ON
```

```
        // ------------------------------------------------------
        //  All constraints back ON
        // ------------------------------------------------------
        $display("\n--- All constraints ON ---\n");

        stim.constraint_mode(1);
        stim.print_constraint_mode();

        repeat (10) begin
            assert(stim.randomize()) else $fatal(1, "randomize() failed");
            stim.drive(A, B, ALU_FUN, enable, RST);
            check(A, B, ALU_FUN, enable, RST);
        end

        // ------------------------------------------------------
        // Req 7a  freeze ALU_FUN, stress one operation
        // ------------------------------------------------------
        $display("\n--- Opcode frozen to ADD ---\n");

        stim.ALU_FUN.rand_mode(0);
        stim.ALU_FUN = ADD;
        stim.print_rand_mode();

        repeat (5) begin
            assert(stim.randomize()) else $fatal(1, "randomize() failed");
            stim.drive(A, B, ALU_FUN, enable, RST);
            check(A, B, ALU_FUN, enable, RST);
        end

        // ------------------------------------------------------
        // freeze A and B, sweep all opcodes
        // ------------------------------------------------------
        $display("\n--- A & B frozen, ALU_FUN free ---\n");

        stim.ALU_FUN.rand_mode(1);
        stim.A.rand_mode(0);
        stim.B.rand_mode(0);
        stim.print_rand_mode();

        repeat (5) begin
            assert(stim.randomize()) else $fatal(1, "randomize() failed");
            stim.drive(A, B, ALU_FUN, enable, RST);
            check(A, B, ALU_FUN, enable, RST);
        end

        // ------------------------------------------------------
        // Final: restore everything, coverage sweep
        // ------------------------------------------------------
        $display("\n--- Final: full randomization, all restored ---\n");

        stim.rand_mode(1);
        stim.constraint_mode(1);

        repeat (20) begin
            assert(stim.randomize()) else $fatal(1, "randomize() failed");
            stim.drive(A, B, ALU_FUN, enable, RST);
            check(A, B, ALU_FUN, enable, RST);
        end

        $display("\n[TB] done\n");
        $stop;
    end
endmodule
```

Do file:

```
if {[file exists work]} { vdel -lib work -all }
vlib work

vlog  *.*v

vsim -sv_seed random work.top

add wave *

run -all
```